

Chapter 12: Basic Maths for DSA

Personal Notes

This is NOT school maths. Here we care about maths only from the programming side: how numbers behave, how digits are extracted, how loops are used on numbers, and how logic gets built around them.

This chapter is the bridge between learning Java syntax and actually solving DSA problems. Once you get strong here, pattern problems, number problems, conditions, and early DSA questions all start to feel natural.

1. Why Basic Maths Matters in Programming

A lot of beginner Java questions are actually built on simple maths ideas:

- Check if a number is even or odd
- Count the digits in a number
- Reverse a number
- Check if a number is palindrome
- Find factorial
- Check if a number is prime
- Find GCD and LCM

These look like "math problems," but in reality they are LOGIC-BUILDING problems. That is exactly why this topic is so important — it teaches you how to translate ideas into code.

2. Number Basics

In programming, you mostly work with two kinds of numbers:

- Integers — whole numbers like 5, 18, 120
- Decimal numbers — like 3.14, 12.5

For beginner DSA and number logic problems, almost everything uses integers.

```
int age = 25;  
int solvedProblems = 150;  
double completionRate = 87.5;
```

For this chapter, the main focus is integer mathematics.

3. Basic Arithmetic Operations

Java supports all the standard math operations using these symbols:

Operation	Symbol
Addition	+
Subtraction	-
Multiplication	*
Division	/
Modulo (remainder)	%

Example

```
int a = 10;
int b = 3;

System.out.println(a + b);    // 13
System.out.println(a - b);    // 7
System.out.println(a * b);    // 30
System.out.println(a / b);    // 3    (NOT 3.333)
System.out.println(a % b);    // 1
```

3.1 Integer Division — A Common Trap

When both operands are integers, Java performs integer division. The decimal part is thrown away — not rounded, just discarded.

```
10 / 3 = 3      (not 3.333...)
5 / 2 = 2       (not 2.5)
7 / 4 = 1       (not 1.75)
```

If you want a decimal result, at least one operand must be a double:

```
double result = 10.0 / 3;    // 3.333...
double result = (double) 10 / 3;    // 3.333... (casting)
```

3.2 The Modulo Operator %

The modulo operator gives the REMAINDER after division.

```
10 % 3 = 1      (because 10 = 3 × 3 + 1)
15 % 4 = 3      (because 15 = 4 × 3 + 3)
20 % 5 = 0      (because 20 is divisible by 5)
```

This operator is everywhere in programming. It is used for:

- Even/odd checks
- Digit extraction (the trick of this entire chapter)
- Cyclic operations (wrap around an array)
- Almost every number logic problem

4. The Digit Extraction Trick

This is the single most important idea in this entire chapter. Memorize it. Almost every number problem uses it.

THE TWO MAGIC OPERATIONS: $n \% 10 \rightarrow$ gives you the LAST digit. $n / 10 \rightarrow$ removes the last digit.

Example with $n = 1234$

```

Step 1:  n = 1234
         n % 10 = 4      <-- last digit
         n / 10 = 123    <-- last digit removed

Step 2:  n = 123
         n % 10 = 3
         n / 10 = 12

Step 3:  n = 12
         n % 10 = 2
         n / 10 = 1

Step 4:  n = 1
         n % 10 = 1
         n / 10 = 0      <-- stop when n becomes 0

```

This single idea is used in:

- Count digits
- Sum of digits
- Reverse a number
- Palindrome number
- Armstrong number
- Counting even/odd digits

Once you internalize it, all those problems suddenly look the same.

5. Problem 1: Count Digits in a Number

Given a positive integer, count how many digits it has. Example: $12345 \rightarrow 5$.

Intuition

Every time we divide a number by 10, one digit gets removed from the right. So if we keep dividing by 10 until the number becomes 0, the number of steps equals the digit count.

Code

```
public class Main {
    public static void main(String[] args) {
        int n = 12345;
        int count = 0;

        while (n > 0) {
            count++;
            n = n / 10;
        }

        System.out.println("Number of digits = " + count);
    }
}
```

Dry Run for n = 12345

```
Start:    n = 12345,  count = 0

Iter 1:   n = 12345 / 10 = 1234    count = 1
Iter 2:   n = 1234  / 10 = 123     count = 2
Iter 3:   n = 123   / 10 = 12      count = 3
Iter 4:   n = 12    / 10 = 1       count = 4
Iter 5:   n = 1     / 10 = 0       count = 5

Loop stops because n = 0.

Output:   Number of digits = 5
```

6. Problem 2: Sum of Digits

Find the sum of all digits of a number. Example: $1234 \rightarrow 1 + 2 + 3 + 4 = 10$.

Intuition

Process digits one by one: extract the last digit using % 10, add it to the sum, then remove the last digit using / 10. Repeat until n becomes 0.

Code

```
public class Main {
    public static void main(String[] args) {
        int n = 1234;
        int sum = 0;
```

```

        while (n > 0) {
            int digit = n % 10;
            sum = sum + digit;
            n = n / 10;
        }

        System.out.println("Sum of digits = " + sum);
    }
}

```

Dry Run for n = 1234

```

Start:    n = 1234,  sum = 0

Iter 1:   digit = 4,  sum = 4,    n = 123
Iter 2:   digit = 3,  sum = 7,    n = 12
Iter 3:   digit = 2,  sum = 9,    n = 1
Iter 4:   digit = 1,  sum = 10,   n = 0

Output:   Sum of digits = 10

```

7. Problem 3: Reverse a Number

Reverse the digits of a number. Example: 1234 → 4321.

Intuition

As we extract digits from right to left, we build a new "reversed" number using this formula:

```
rev = rev * 10 + digit;
```

Why multiply by 10? Because it shifts the old digits one place to the left, creating an empty spot for the new digit.

Code

```

public class Main {
    public static void main(String[] args) {
        int n = 1234;
        int rev = 0;

        while (n > 0) {
            int digit = n % 10;
            rev = rev * 10 + digit;
            n = n / 10;
        }

        System.out.println("Reversed number = " + rev);
    }
}

```

```
}
```

Dry Run for n = 1234

```
Start:    n = 1234,  rev = 0

Iter 1:   digit = 4,  rev = 0*10 + 4  = 4      n = 123
Iter 2:   digit = 3,  rev = 4*10 + 3  = 43     n = 12
Iter 3:   digit = 2,  rev = 43*10 + 2  = 432    n = 1
Iter 4:   digit = 1,  rev = 432*10 + 1 = 4321   n = 0

Output:   Reversed number = 4321
```

8. Even and Odd Numbers

A number is even if it is divisible by 2, and odd otherwise. So the check is simple:

```
n % 2 == 0    // true means even
```

Code

```
int n = 7;

if (n % 2 == 0) {
    System.out.println("Even");
} else {
    System.out.println("Odd");
}
```

Here $7 \% 2 = 1$, which is not 0, so the output is "Odd".

9. Problem 4: Palindrome Number

A number is a palindrome if it reads the same forward and backward. For example: 121, 1331, 12321 are palindromes. 123, 145 are not.

Intuition

Reverse the number using the same trick as before, then compare with the original. If they match, it is a palindrome.

Code

```
public class Main {
    public static void main(String[] args) {
        int n = 121;
        int original = n;
        int rev = 0;

        while (n > 0) {
```

```

        int digit = n % 10;
        rev = rev * 10 + digit;
        n = n / 10;
    }

    if (original == rev) {
        System.out.println("Palindrome");
    } else {
        System.out.println("Not Palindrome");
    }
}
}

```

Important: Notice the line `int original = n;` at the top. Since the loop modifies `n`, we **MUST** save the original value first. Forgetting this is one of the most common beginner mistakes in number problems.

10. Problem 5: Prime Number

A prime number is a number greater than 1 that is divisible only by 1 and itself. Examples: 2, 3, 5, 7, 11, 13 are prime. 4, 6, 8, 9, 10 are not.

Basic Intuition

To check if `n` is prime, try dividing it by every number from 2 to `n-1`. If any one of them divides `n` exactly, `n` is NOT prime.

Basic Code

```

public class Main {
    public static void main(String[] args) {
        int n = 7;
        boolean isPrime = true;

        if (n <= 1) {
            isPrime = false;
        } else {
            for (int i = 2; i < n; i++) {
                if (n % i == 0) {
                    isPrime = false;
                    break;
                }
            }
        }

        if (isPrime) {
            System.out.println("Prime");
        } else {
            System.out.println("Not Prime");
        }
    }
}

```

```

    }
}

```

Optimized Version (Very Important)

You only need to check divisors up to $\sqrt{n}$. Why? Because factors come in pairs — if $n = a \times b$ and $a \leq b$, then a must be $\leq \sqrt{n}$. So if no divisor exists up to $\sqrt{n}$, none exists at all.

```

for (int i = 2; i * i <= n; i++) {
    if (n % i == 0) {
        isPrime = false;
        break;
    }
}

```

This tiny change makes prime checking dramatically faster for large numbers — and it's a frequently asked interview detail.

Why $i * i$ instead of `Math.sqrt(n)`? Squaring i is faster than computing a square root, and $i * i \leq n$ is mathematically the same as $i \leq \sqrt{n}$. Both work — using $i * i$ is just a small optimization.

11. Problem 6: Factorial

The factorial of n (written $n!$) is the product of all integers from 1 to n . Example: $5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$.

Code

```

public class Main {
    public static void main(String[] args) {
        int n = 5;
        int fact = 1;

        for (int i = 1; i <= n; i++) {
            fact = fact * i;
        }

        System.out.println("Factorial = " + fact);
    }
}

```

Dry Run

```

Start:    fact = 1

i = 1 → fact = 1    * 1 = 1
i = 2 → fact = 1    * 2 = 2
i = 3 → fact = 2    * 3 = 6
i = 4 → fact = 6    * 4 = 24

```



```
i = 5 → fact = 24 * 5 = 120
```

```
Output: Factorial = 120
```

Watch out for overflow: Factorial grows extremely fast. 13! is already too big for int. For larger factorials, use long. For really large ones (20! and beyond), you need BigInteger.

12. Problem 7: GCD (Greatest Common Divisor)

The GCD of two numbers is the largest number that divides both. Example: $\text{GCD}(12, 18) = 6$ because 6 is the biggest number that divides both 12 and 18.

Best Method: Euclidean Algorithm

This is the standard, efficient way to find GCD. The rule is:

```
GCD(a, b) = GCD(b, a % b)
```

Keep replacing (a, b) with (b, a % b) until b becomes 0.

When b = 0, a holds the GCD.

Code

```
public class Main {
    public static void main(String[] args) {
        int a = 12;
        int b = 18;

        while (b != 0) {
            int temp = b;
            b = a % b;
            a = temp;
        }

        System.out.println("GCD = " + a);
    }
}
```

Dry Run for a = 12, b = 18

Start: a = 12, b = 18

Iter 1: temp = 18, b = 12 % 18 = 12, a = 18
State: a = 18, b = 12

Iter 2: temp = 12, b = 18 % 12 = 6, a = 12
State: a = 12, b = 6

```
Iter 3:   temp = 6,   b = 12 % 6 = 0,   a = 6  
        State: a = 6, b = 0
```

Loop stops because $b = 0$.

Output: GCD = 6

13. Problem 8: LCM (Least Common Multiple)

LCM is the smallest number that is divisible by both. Example: $\text{LCM}(12, 18) = 36$.

Magic Formula

$$\text{LCM} \times \text{GCD} = a \times b$$

so

$$\text{LCM} = (a \times b) / \text{GCD}$$

This means you don't need a separate algorithm — just compute GCD using the Euclidean algorithm, then divide.

Code

```
public class Main {  
    public static void main(String[] args) {  
        int a = 12;  
        int b = 18;  
  
        int x = a;  
        int y = b;  
  
        // find GCD  
        while (y != 0) {  
            int temp = y;  
            y = x % y;  
            x = temp;  
        }  
        int gcd = x;  
  
        // use the formula  
        int lcm = (a * b) / gcd;  
  
        System.out.println("LCM = " + lcm);  
    }  
}
```

For 12 and 18: $\text{GCD} = 6$, so $\text{LCM} = (12 \times 18) / 6 = 36$.

14. Problem 9: Armstrong Number

An Armstrong number is one where the sum of its digits raised to the power of (number of digits) equals the original number.

For 3-digit numbers, this is: $abc = a^3 + b^3 + c^3$.

Example: $153 = 1^3 + 5^3 + 3^3$
 $= 1 + 125 + 27$
 $= 153$ ✓ Armstrong number

Counter: $123 = 1^3 + 2^3 + 3^3$
 $= 1 + 8 + 27$
 $= 36$ ✗ Not Armstrong

Code (3-digit case)

```
public class Main {
    public static void main(String[] args) {
        int n = 153;
        int original = n;
        int sum = 0;

        while (n > 0) {
            int digit = n % 10;
            sum = sum + (digit * digit * digit);
            n = n / 10;
        }

        if (sum == original) {
            System.out.println("Armstrong Number");
        } else {
            System.out.println("Not Armstrong Number");
        }
    }
}
```

15. Problem 10: Power of a Number

Compute a^b — that is, a raised to the power b . Example: $2^5 = 32$.

Intuition

Just multiply a by itself, b times. Start with ans = 1 and multiply b times.

Code

```
public class Main {
    public static void main(String[] args) {
        int a = 2;
    }
}
```

```

    int b = 5;
    int ans = 1;

    for (int i = 1; i <= b; i++) {
        ans = ans * a;
    }

    System.out.println("Power = " + ans);
}
}

```

Dry Run for 2^5

Start: ans = 1

```

i = 1: ans = 1 * 2 = 2
i = 2: ans = 2 * 2 = 4
i = 3: ans = 4 * 2 = 8
i = 4: ans = 8 * 2 = 16
i = 5: ans = 16 * 2 = 32

```

Output: Power = 32

Advanced version: For very large b , this $O(b)$ loop is too slow. Fast exponentiation using divide-and-conquer ($a^b = a^{(b/2)} \times a^{(b/2)}$) brings it down to $O(\log b)$. You'll learn this when you reach recursion.

16. Problem 11: Perfect Number

A perfect number is one whose proper divisors (divisors excluding the number itself) add up to the number.

```

Example: 6  →  divisors (excluding 6):  1, 2, 3
           sum = 1 + 2 + 3 = 6           ✓ Perfect

          28 →  divisors:  1, 2, 4, 7, 14
           sum = 1 + 2 + 4 + 7 + 14 = 28 ✓ Perfect

          10 →  divisors:  1, 2, 5
           sum = 1 + 2 + 5 = 8           ✗ Not Perfect

```

Code

```

public class Main {
    public static void main(String[] args) {
        int n = 6;
        int sum = 0;

        for (int i = 1; i < n; i++) {

```

```

        if (n % i == 0) {
            sum = sum + i;
        }
    }

    if (sum == n) {
        System.out.println("Perfect Number");
    } else {
        System.out.println("Not Perfect Number");
    }
}
}

```

Logic: check all numbers from 1 to $n-1$. If a number divides n exactly, it is a proper divisor — add it to the sum. At the end, compare the sum with n .

17. Problem 12: Count Even Digits

Count how many digits in a number are even. Example: 248531 → even digits are 2, 4, 8 → count = 3.

Code

```

public class Main {
    public static void main(String[] args) {
        int n = 248531;
        int count = 0;

        while (n > 0) {
            int digit = n % 10;

            if (digit % 2 == 0) {
                count++;
            }

            n = n / 10;
        }

        System.out.println("Even digits count = " + count);
    }
}

```

Same digit-extraction trick as before, plus one extra check: $\text{digit} \% 2 == 0$ for each digit.

18. Problem 13: Print All Primes from 1 to N

Print every prime number between 1 and a given n . This combines the prime check with a loop over the whole range.

Code

```
public class Main {
    public static void main(String[] args) {
        int n = 20;

        for (int num = 2; num <= n; num++) {
            boolean isPrime = true;

            for (int i = 2; i * i <= num; i++) {
                if (num % i == 0) {
                    isPrime = false;
                    break;
                }
            }

            if (isPrime) {
                System.out.print(num + " ");
            }
        }
    }
}
```

Output:

```
2 3 5 7 11 13 17 19
```

Notice the use of `i * i <= num` — the optimized prime check from earlier.

19. Core Patterns You Must Remember

After this chapter, you should know these patterns by heart. They are not just formulas — they are coding tools you'll use forever.

Pattern	What it does
<code>n % 10</code>	Extract the last digit of <code>n</code>
<code>n / 10</code>	Remove the last digit of <code>n</code>
<code>n % 2 == 0</code>	Check if <code>n</code> is even
<code>rev = rev * 10 + digit</code>	Build the reversed number digit by digit
<code>while (n > 0)</code>	Loop through all digits of <code>n</code>
<code>GCD(a, b) = GCD(b, a % b)</code>	Euclidean algorithm for GCD
<code>LCM = (a × b) / GCD</code>	LCM via GCD (avoids a separate algorithm)
<code>i * i <= n</code>	Loop till $\sqrt{n}$ (prime check optimization)

Pattern	What it does
<code>int original = n;</code>	Always save the original before modifying

20. Common Beginner Mistakes

Mistake 1: Forgetting that integer division drops decimals

```
int avg = 5 / 2;           // 2, NOT 2.5
double avg = 5 / 2;       // STILL 2.0 – Java did int division first
double avg = 5.0 / 2;     // 2.5 – at least one operand is double
```

Mistake 2: Using % without thinking about remainder

Always mentally connect % to "what is left over after division."

```
17 % 5 = 2    because 17 = 5 × 3 + 2
20 % 4 = 0    because 20 = 4 × 5 + 0
9 % 7 = 2     because 9 = 7 × 1 + 2
```

Mistake 3: Losing the original number

Inside the digit-extraction loop, `n` gets destroyed (becomes 0). If you need it later — for palindrome checks, Armstrong checks, etc. — save it FIRST.

```
int original = n; // save before the loop modifies n
while (n > 0) {
    // process digits...
    n = n / 10;
}
// now we can still compare with 'original'
```

Mistake 4: Forgetting to update `n` inside the loop

This is the classic infinite loop. `n` never changes, so the loop never ends.

```
while (n > 0) {
    int digit = n % 10;
    sum += digit;
    // FORGOT: n = n / 10;
}
// INFINITE LOOP – your program will hang
```

Mistake 5: Off-by-one in prime checking

Starting the loop at 1 instead of 2 will make every number look non-prime (because every number is divisible by 1). Always start at `i = 2`.

Mistake 6: Integer overflow with factorial / power

Factorial and power grow VERY fast. 13! overflows int. Use long when needed:

```
long fact = 1;
```

21. Practice Advice

Do not just READ this chapter. The only way to truly learn it is to dry-run the code by hand.

On paper, work through every problem with these inputs:

- $n = 1234$ (general)
- $n = 121$ (palindrome)
- $n = 153$ (Armstrong)
- $n = 28$ (perfect)
- $a = 36, b = 24$ (GCD/LCM)

Track every variable, every iteration. That is where real understanding develops — not from reading, but from doing.

22. Extra Practice Problems

Try these on your own using the patterns from this chapter:

1. Count the number of odd digits in a number.
2. Find the largest digit in a number.
3. Find the smallest digit in a number.
4. Print the multiplication table of n (1 to 10).
5. Sum of squares of digits (e.g., $123 \rightarrow 1^2 + 2^2 + 3^2 = 14$).
6. Check if a number is a happy number.
7. Print the Fibonacci series up to n terms.
8. Check if a number is a strong number (sum of factorial of digits).
9. Generalize Armstrong check for any number of digits.
10. Find the next prime number after n .

23. Final Takeaway

Basic maths in programming is not about memorizing formulas. It is about understanding:

- How numbers break apart digit by digit
- How loops interact with numbers
- How conditions are used to build logic
- How simple ideas combine into real solutions

Once you become comfortable with these problems, you start to feel something change — you are no longer just writing Java syntax. You are actually solving problems.

That mental shift is what separates someone learning to code from someone who can code.

Key Takeaway: Memorize the digit-extraction trick ($n \% 10$ and $n / 10$), the Euclidean GCD pattern, and the prime-check optimization ($i * i \leq n$). These three ideas show up everywhere in DSA — pattern problems, number theory, hashing, even some graph problems. Master these, and DSA stops feeling scary.

24. What's Next?

Now that maths and number logic feel comfortable, the natural next steps are:

- Recursion — solving problems by breaking them into smaller versions of themselves
- Bit manipulation — working with binary representations of numbers
- Searching algorithms (Linear and Binary Search)
- Sorting algorithms (Bubble, Selection, Insertion, Merge, Quick)
- Time and space complexity analysis
- Two-pointer and sliding window techniques